

Atelier 4 : les requêtes de la clinique

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Base de données 1, fondements, modélisation et SQL · **Chapitre** : 4, Interroger les données en SQL · **Version** : 2026-10-31

Avant de commencer

Ce que vous allez construire	Le « tableau de bord » SQL de la clinique Santé Plus : une vingtaine de requêtes qui répondent aux questions de l'accueil, des médecins, de la caisse et de la direction, une vue et un index
Durée	🟢 Débutant : 7 h · 🔵 Standard : 5 h · 🟣 Expert : 3 h 30 (sans les défis)
Prérequis	Avoir lu le chapitre 4
Fichiers fournis	`clinique-complete.sql` (la base de référence) et `atelier4-archive-rdv.sql` (100 000 rendez-vous archivés, pour l'étape 6)
Rendu	Le fichier `atelier4.sql` : chaque requête précédée de la question en commentaire et suivie, en commentaire, du nombre de lignes obtenues

> 📍 **Point de départ : la base de référence.** Exécutez `clinique-complete.sql` dans `clinique` : vos résultats doivent être exactement ceux de cet énoncé. (Votre propre base de l'atelier 3 n'est pas perdue : gardez votre script `clinique.sql`.)
>
> **La suite** : le mini-projet de synthèse, sur un sujet de votre choix.

Comment travailler

L'atelier est découpé en **8 étapes** (0 à 7), de la plus simple à la plus complète. Pour chaque question : écrivez la requête, exécutez-la, **comparez** avec le résultat attendu. Si ça ne correspond pas, ne passez pas à la suite : cherchez l'erreur.

Trois règles pour tout l'atelier

1. **Un `ORDER BY`** dans chaque requête qui renvoie plusieurs lignes : sans lui, l'ordre n'est pas garanti.
2. **Des alias** de tables dans toutes les jointures (`c`, `p`, `m`…).

3. **Une requête à la fois** : construisez-la morceau par morceau (d'abord le `FROM` et les jointures avec `SELECT *`, puis le `WHERE`, puis le `GROUP BY`...).

Étape 0 : charger la base (10 min)

🎯 **Objectif** : partir des mêmes données que tout le monde.

🛠️ **A faire** : exécutez `clinique-complete.sql`, puis `SELECT COUNT(*) FROM patient;` et `SELECT COUNT(*) FROM consultation;`.

✅ **Vérification** : 9 patients, 10 consultations.

Étape 1 : l'accueil (30 min)

🎯 **Objectif** : des sélections simples.

🛠️ **A faire**

1. Les patients nés **en 2000 ou après**, du plus âgé au plus jeune (nom, prénom, date de naissance).
2. Les consultations **du 10 au 20 octobre 2026** inclus, par date (numéro, date, motif).
3. Les noms des médicaments en **comprimé**, par ordre alphabétique.
4. Les consultations dont le motif **commence par** « Contrôle ».

🔎 **A chercher** : chapitre 4, sections 1.3 et 1.4.

💡 **A comprendre**

- Pour la question 2, `BETWEEN` inclut-il le 20 octobre ?
- Pour la question 4, quelle différence entre `LIKE 'Contrôle%'` et `LIKE '%Contrôle%'` ?

✅ **Vérification**

Question	Résultat attendu
1	4 lignes : Ndiaye Aliou (2001), Cissé Bineta (2005), Sarr Fatou (2019), Diallo Ousmane (2021)
2	5 lignes : consultations 3, 4, 5, 6, 7
3	5 lignes : Amlodipine, Artéméther-luméfantrine, Fer et acide folique, Ibuprofène, Paracétamol
4	2 lignes : consultations 3 (Contrôle tension) et 8 (Contrôle)

Étape 2 : les fonctions (30 min)

🎯 **Objectif** : transformer et classer les valeurs.

🛠️ **A faire**

1. Pour chaque patient, son nom, son prénom, et sa catégorie : `enfant` s'il est né le 1er janvier 2014 ou après, `adulte` sinon.
2. Pour chaque séjour **terminé**, son numéro et son nombre de nuits (avec la bonne fonction pour **votre** SGBD).
3. Pour chaque patient, son nom et son assurance, ou `sans assurance` s'il n'en a pas.

🔎 **A chercher** : chapitre 4, section 2.

💡 **A comprendre**

- Sur MySQL, que donnerait `date_sortie - date_entree` pour un séjour du 28 octobre au 2 novembre ? Pourquoi ?

✅ **Vérification** : question 1 : 2 enfants (Fatou Sarr, Ousmane Diallo) et 7 adultes ; question 2 : séjour 1, 3 nuits ; séjour 2, 5 nuits ; question 3 : 9 lignes, dont 5 « sans assurance ».

Étape 3 : la direction compte (40 min)

🎯 **Objectif** : agréger et regrouper.

🛠️ **A faire**

1. Le nombre de consultations **par service** (nom du service), du plus grand au plus petit.
2. Les jours où il y a eu **au moins deux** consultations, avec leur nombre et le total des tarifs.
3. Le tarif moyen **par spécialité**, arrondi à l'unité, du plus cher au moins cher.

🔎 **A chercher** : chapitre 4, sections 3 et 4.1.

💡 **A comprendre**

- Pour la question 1, combien de tables faut-il joindre ? Pourquoi passer par `medecin` ?
- Pour la question 2, pourquoi la condition « au moins deux » ne peut-elle pas aller dans le `WHERE` ?

✅ **Vérification**

Question	Résultat attendu
1	Consultations générales 4, Pédiatrie 3, Cardiologie 2, Maternité 1
2	1 ligne : 2026-10-05, 2 consultations, 25000
3	Cardiologie 20000, Gynécologie 15000, Pédiatrie 15000, Médecine générale 10000

Étape 4 : les médecins et les hospitalisations (45 min)

🎯 **Objectif** : joindre trois tables ou plus, et garder ce qui n'a pas de correspondance.

🛠️ **A faire**

1. Les ordonnances des consultations 5 et 7 : numéro de consultation, nom du patient, nom du médicament, posologie.
2. Tous les séjours avec le nom du patient, le numéro de la chambre, le nom du service de la chambre et son prix par nuit.
3. Les patients **actuellement hospitalisés** (séjour sans date de sortie), avec leur chambre et leur date d'entrée.
4. **Toutes** les chambres, avec leur nombre de séjours (0 compris).

🔎 **A chercher** : chapitre 4, sections 4.1 et 4.2.

💡 **A comprendre**

- Pour la question 4, que se passe-t-il avec un `JOIN` au lieu d'un `LEFT JOIN` ? Et avec `COUNT(*)` au lieu de `COUNT(se.num_sejour)` ?

✅ **Vérification**

Question	Résultat attendu
1	4 lignes : consultation 5 (Diallo : Paracétamol, Sels de réhydratation orale) ; consultation 7 (Fall : Artéméther-luméfantrine, Paracétamol)
2	3 lignes : séjour 1 (Diallo, P-12, Pédiatrie, 15000), 2 (Thiam, C-21, Cardiologie, 35000), 3 (Sy, M-13, Maternité, 20000)
3	1 ligne : Sy Mariama, chambre M-13, entrée le 2026-10-27
4	C-21 : 1, G-01 : 0, M-13 : 1, P-11 : 0, P-12 : 1

Étape 5 : les questions difficiles (45 min)

🎯 **Objectif** : sous-requêtes et opérateurs ensemblistes.

🛠️ **A faire**

1. Les médecins qui n'ont suivi **aucun** séjour.
2. Les patients qui ont eu **au moins une** consultation plus chère que la moyenne des consultations (sans doublon).
3. Les patients qui **n'ont encore rien payé** (aucun paiement sur aucune de leurs consultations), avec `NOT EXISTS`.
4. Les villes qui ont des patients, mais **aucun** patient hospitalisé, avec `EXCEPT` (`MINUS` sur Oracle).

🔎 **A chercher** : chapitre 4, sections 5 et 6.

💡 **A comprendre**

- Pour la question 1, la requête avec `NOT IN` est-elle sûre ici ? Que faudrait-il vérifier sur la colonne `sejour.num_medecin` ? (Encadré ⚠️ de la section 5.2.)
- Pour la question 3, Bineta Cissé n'a aucune consultation : pourquoi apparaît-elle quand même ?

✅ **Vérification**

Question	Résultat attendu
1	Ba Mamadou, Faye Aminata, Gueye Cheikh
2	5 lignes : Diallo Ousmane, Ndiaye Moussa, Sarr Fatou, Sy Mariama, Thiam Khady
3	Ndiaye Aliou, Cissé Bineta
4	Rufisque, Saint-Louis

Étape 6 : une vue et un index (40 min)

🎯 **Objectif** : enregistrer un calcul métier, et accélérer une recherche.

🛠️ **A faire**

1. Créez la vue `v_reste_patient` : pour chaque patient qui a eu des consultations, son numéro, son nom, son prénom et le **reste à payer** (somme des tarifs de ses consultations moins la somme de leurs paiements).
2. Interrogez-la : les patients qui doivent encore de l'argent, du plus gros reste au plus petit.
3. Exécutez le fichier fourni `atelier4-archive-rdv.sql`. Affichez le plan d'exécution de la recherche des rendez-vous du patient **4321**, créez l'index qui convient, et affichez à nouveau le plan.

🔎 **A chercher** : chapitre 4, sections 7 et 8.

💡 **A comprendre**

- Pour la vue, pourquoi faut-il **d'abord** additionner les paiements par consultation (dans une sous-requête), avant de joindre ? (Essayez sans : Moussa Ndiaye, patient n°2, a payé sa consultation de 20000 FCFA en deux fois ; que devient son reste à payer ?)
- Combien de lignes le SGBD estime-t-il examiner avant et après l'index ?

✅ **Vérification**

Contrôle	Résultat attendu
Vue, patients qui doivent de l'argent	Sarr Fatou 15000, Ndiaye Aliou 10000, Fall Ibrahima 5000, Sy Mariama 3000, Diop Awa 2000

Contrôle	Résultat attendu
Plan avant l'index	Lecture de toute la table (`Seq Scan` sur PostgreSQL ; `type: ALL` sur MySQL)
Plan après l'index	Passage par votre index (`Index Scan` ou `Bitmap Index Scan` ; `type: ref`) ; 20 lignes

▼ Indice : la structure de la vue

`FROM patient p JOIN consultation c ON ... LEFT JOIN (SELECT num_consultation, SUM(montant) AS total FROM paiement GROUP BY num_consultation) pa ON ...` , puis `GROUP BY` sur le patient et `SUM(c.tarif) - COALESCE(SUM(pa.total), 0)` .

Étape 7 : vos questions (30 min)

🎯 **Objectif** : passer de « je recopie » à « je conçois ».

🛠️ **A faire** : inventez **trois** questions utiles à la clinique, qui ne sont pas dans cet énoncé, et écrivez leurs requêtes. Au moins une avec une jointure de trois tables, au moins une avec un `GROUP BY` … `HAVING` .

✅ **Vérification** : chaque question est écrite en français au-dessus de sa requête, et le résultat y répond vraiment.

🏆 Défis (facultatifs)

1. **Le classement des médecins.** Pour chaque médecin : nom, nombre de consultations, chiffre d'affaires, et son rang (cherchez « SQL RANK OVER »).
2. **La facture d'un séjour.** Pour chaque séjour terminé : nombre de nuits × prix par nuit, et la part prise en charge par l'assurance du patient.
3. **La chambre double.** La requête du défi 3 de l'atelier 3 : les chambres et le nombre de patients présents en ce moment.
4. **Sur Oracle.** Exécutez vos requêtes de l'étape 5 sur FreeSQL (après y avoir chargé la base) : lesquelles faut-il modifier ?

Grille d'autoévaluation

Je sais…	Oui	Pas encore
filtrer et trier avec `WHERE` , `BETWEEN` , `IN` , `LIKE` , `IS NULL` , `ORDER BY`	☐	☐
utiliser des fonctions (textes, dates, `CASE` , `COALESCE`)	☐	☐

Je sais…	Oui	Pas encore
regrouper avec `GROUP BY` et filtrer les groupes avec `HAVING`	☐	☐
joindre trois tables ou plus en suivant les clés étrangères	☐	☐
choisir entre `JOIN` et `LEFT JOIN`	☐	☐
écrire une sous-requête avec `IN`, `NOT EXISTS`, ou corrélée	☐	☐
créer une vue et un index, et lire un plan d'exécution simple	☐	☐